

Mengukur Penundaan End-to-End pada Platform IoT SDN Berenergi Rendah

Mykola Beshley¹, Natalia Kryvinska^{2,*}, Halyna Beshley¹, Orest Kochan¹ dan Leonard Barolli³

¹Departemen Telekomunikasi, Universitas Nasional Politeknik Lviv, 79013 Lviv, Ukraina

²Departemen Sistem Informasi, Fakultas Manajemen, Universitas Comenius di Bratislava, 82005 Bratislava, Slovakia

³Departemen Teknik Informasi dan Komunikasi, Fakultas Teknik Informatika, Institut Teknologi Fukuoka (FIT), 3-30-1

Wajiro-Higashi, Higashi-Ku, Fukuoka 811-0295, Jepang *Penulis Koresponden:

Natalia Kryvinska. Email: natalia.kryvinska@uniba.sk Diterima: 12 Maret 2021;

Diterima: 17 April 2021

Abstrak: Dalam tulisan ini, kami mengembangkan platform Internet of Things (IoT) berbasis Software Defined Networking (SDN) hemat energi baru yang dapat dikonfigurasi ulang sesuai dengan kebutuhan aplikasi IoT target.

Secara teknis, platform ini terdiri dari seperangkat komputer papan tunggal berbiaya rendah dan hemat energi, yang saling terhubung dalam jaringan dengan konfigurasi yang ditentukan perangkat lunak. Sakelar SDN yang diusulkan diterapkan pada papan Raspberry Pi 3 menggunakan perangkat lunak Open vSwitch (OvS), sedangkan pengontrol Floodlight diterapkan pada papan Orange Pi Prime. Kami pertama kali mempresentasikan dan menerapkan metode untuk mengukur penundaan yang diperkenalkan oleh setiap komponen infrastruktur IoT, mulai dari sensor, inti SDN, broker IoT, hingga pelanggan IoT. Oleh karena itu, kami menyajikan pendekatan untuk memperkirakan efisiensi energi untuk platform IoT berbasis SDN yang sebanding dengan lalu lintas.

Eksperimen yang dilakukan pada topologi SDN nyata berdasarkan komputer papan tunggal menunjukkan bahwa pendekatan kami tidak hanya menghemat hingga 53,56% energi pada intensitas lalu lintas rendah, namun juga memberikan jaminan QoS untuk aplikasi IoT.

Kata Kunci: Internet of Things; jaringan yang ditentukan perangkat lunak; aliran terbuka; buka vswitch; raspberry-pi

1. Perkenalan

1.1 Latar Belakang dan Rumusan Masalah

Di dunia modern, Internet telah menjadi kebutuhan dasar setiap orang. Efisiensi energi dan infrastruktur jaringan ramah lingkungan sangat diminati karena mengurangi biaya energi sehingga menghemat uang konsumen. Efisiensi energi mengurangi polusi lingkungan, menyediakan lebih banyak energi, dan meningkatkan perekonomian. Dalam makalah [1–4] Algoritma Optimasi Paus-Optimasi Api Ngengat hibrida diusulkan untuk memilih Cluster Head yang optimal, yang pada gilirannya mengoptimalkan konsumsi energi untuk jaringan IoT. Pendekatan ini memberikan penyeimbangan beban yang efisien, energi sisa yang lebih baik, dan latensi yang lebih pendek, sehingga menghasilkan masa pakai jaringan yang lebih lama. Salah satu tantangan terpenting adalah menjaga masa pakai baterai pada perangkat yang digunakan di semua jaringan IoT. Karena banyak perangkat IoT



Karya ini dilisensikan di bawah Lisensi Internasional Creative Commons Attribution 4.0, yang mengizinkan penggunaan, distribusi, dan reproduksi tanpa batas dalam media apa pun, asalkan karya asli dikutip dengan benar.

tidak mengisi ulang, penulis di [5] mengusulkan pendekatan untuk menghemat masa pakai baterai jaringan IoT menggunakan prediksi masa pakai baterai awal.

Dalam makalah [5], Anwaar membuktikan bahwa Raspberry Pi mengkonsumsi lebih sedikit daya dan dapat menghemat sejumlah besar energi saat melakukan tugas komputasi rutin. Dalam makalah [6], penulis menyarankan alternatif yang lebih hemat biaya untuk mengimplementasikan SDN testbed dengan Open vSwitch (OvS), berdasarkan komputer mini Raspberry-Pi berenergi rendah dengan kemampuan program yang mudah. Testbed ini diusulkan untuk jaringan skala kecil. Namun penulis tidak melakukan penelitian mengenai kualitas pemberian layanan. Para penulis di [7] mengembangkan prototipe saklar SDN yang hemat biaya, terukur, dan berkemampuan QoS untuk komunikasi IoT berdasarkan Raspberry Pi 3. Namun, penulis tidak melakukan penelitian tentang kesesuaian platform tersebut untuk misi. -aplikasi IoT penting sesuai dengan persyaratan QoS yang dapat diterima. Mereka juga tidak mempertimbangkan pendekatan perutean SDN dalam kondisi beban jaringan yang tinggi dengan adanya lalu lintas IoT multilayanan dengan persyaratan QoS yang berbeda. Karena tingginya biaya dan kompleksitas dari perangkat keras switch SDN yang ada, penulis di [8] membuat switch SDN yang hemat biaya menggunakan Raspberry Pi untuk menguji ide-ide mereka. Para penulis di [9] menyarankan penggunaan komputer papan tunggal yang hemat biaya dalam kombinasi dengan layanan jaringan kemas yang dapat diskalakan (misalnya, VOIP, layanan web, dll.) menggunakan SDN untuk menyediakan manajemen perangkat terpusat. Konfigurasi prototipe berdasarkan Raspberry Pi dijelaskan secara rinci dan pengujian operasional awal disajikan sebagai sarana untuk memastikan kelayakan teknologi konsep dalam berbagai konfigurasi topologi. Konsumsi energi peralatan jaringan dan pemanfaatan saluran bervariasi tergantung pada beban dan struktur lalu lintas jaringan. Integrasi bidang kontrol dan data ke dalam perangkat individual dalam sistem jaringan tradisional memerlukan lebih banyak pemrosesan, sehingga menghasilkan konsumsi energi yang lebih tinggi dibandingkan perangkat SDN. Akibatnya, jaringan IP tradisional memiliki lebih banyak masalah konsumsi energi karena kompleksitasnya dalam manajemen lalu lintas, pengendalian, dan pengoperasian [10-13].

Untuk aplikasi IoT yang sangat penting, infrastruktur jaringan merupakan komponen ekosistem yang penting. Untuk menyediakan QoS yang sulit untuk aplikasi IoT, penting untuk memastikan mekanisme yang sesuai di setiap lapisan infrastruktur IoT, karena beberapa faktor, seperti penundaan E2E, sangat penting [13-16]. Penundaan pada lapisan mana pun dapat mengakibatkan QoS yang tidak dapat diterima untuk aplikasi keselamatan penting, seperti sistem penggerak otomatis yang selalu membutuhkan umpan balik untuk mempertahankan kendali. Hasil akhir tersebut adalah alasan utama mengapa kepatuhan yang ketat terhadap persyaratan QoS diperlukan dalam IoT yang sangat penting. Hal ini juga merupakan salah satu alasan mengapa perancangan platform IoT yang sangat penting sangatlah rumit.

Saat ini, terdapat banyak solusi SDN, pengontrol, dan berbagai protokol terkenal untuk komunikasi jaringan. Namun, sebagian besar solusi ini mahal dan rumit. Selain itu, mereka tidak selalu tersedia dan sesuai untuk tugas-tugas aplikasi kecil dan menengah, khususnya dalam bisnis [17]. Studi terbaru [18-21] menunjukkan bahwa keputusan untuk mengatur jaringan berdasarkan platform mikrokontroler akan menjaga efisiensi energi, keandalan, dan keamanan jaringan, dan khususnya integrasi dengan Internet tidak memerlukan banyak usaha.

Keserbagunaan mikrokontroler juga terletak pada model Kotak Putih, yang menunjukkan bahwa solusi perangkat keras tidak terikat dengan solusi perangkat lunak, dan akan memungkinkan seseorang dengan mudah berpindah ke satu atau solusi perangkat lunak lainnya sehubungan dengan relevansi dan kebutuhan. Setiap elemen jaringan dapat mengubah tujuan dan perannya dalam jaringan kapan saja.

1.2 Motivasi

Setelah menganalisis karya ilmiah, kami belum menemukan pendekatan yang memungkinkan memperkirakan penundaan yang ditimbulkan oleh setiap komponen infrastruktur IoT, mulai dari sensor,

inti SDN, broker IoT, dan diakhiri dengan pelanggan layanan IoT. Pendekatan ini akan memungkinkan identifikasi hambatan infrastruktur yang menyebabkan degradasi parameter QoS dan membuat keputusan tentang cara mengatasi hambatan ini. Karena biaya energi berkontribusi signifikan terhadap biaya jaringan secara keseluruhan, efisiensi energi merupakan persyaratan desain yang penting untuk platform jaringan modern. Namun, merancang solusi hemat energi merupakan suatu tantangan karena terdapat trade-off antara efisiensi energi dan kinerja jaringan.

1.3 Kontribusi Kami

Untuk mengatasi tantangan-tantangan yang disebutkan di atas, makalah ini berfokus pada pembuatan platform baru yang hemat biaya dan hemat energi yang dapat disesuaikan berdasarkan arsitektur SDN yang dapat menyesuaikan konfigurasinya untuk memenuhi persyaratan QoS pada aplikasi IoT target. Kontribusi utama kami dalam makalah ini dirangkum sebagai berikut:

- Kami mengembangkan prototipe saklar SDN yang berbiaya rendah dan fleksibel serta pengontrol berdasarkan komputer papan tunggal Raspberry Pi 3 dan orange Pi Prime untuk komunikasi IoT;
- Kami merealisasikan broker IoT dan klien web untuk platform IoT berbasis SDN;
- Kami mengembangkan metode untuk mengukur penundaan E2E pada platform IoT berbasis SDN dan sistem pemantauan kualitas pengoperasian platform;
- Kami mempresentasikan bukti konsep platform IoT berbasis SDN yang menggunakan energi komputer papan tunggal yang efisien dan hemat biaya;
- Kami mempresentasikan pendekatan efisiensi energi untuk platform IoT berbasis SDN secara proporsional untuk lalu lintas.

1.4 Organisasi Kertas

Makalah ini disusun sebagai berikut. Bagian 1 menyajikan latar belakang, rumusan masalah dan tinjauan singkat karya terkait. Bagian 2 menjelaskan usulan arsitektur platform IoT berbasis SDN. Bagian 3 menjelaskan metode baru untuk mengukur penundaan E2E pada platform IoT berbasis SDN dan sistem pemantauan. Bagian 4 menjelaskan usulan pendekatan efisiensi energi proporsional untuk Platform IoT Berbasis SDN. Terakhir, kami menarik kesimpulan di Bagian 5.

2 Platform IoT Berbasis SDN yang Hemat Energi dan Bersaing Biaya 2.1

Arsitektur Platform IoT Berbasis SDN

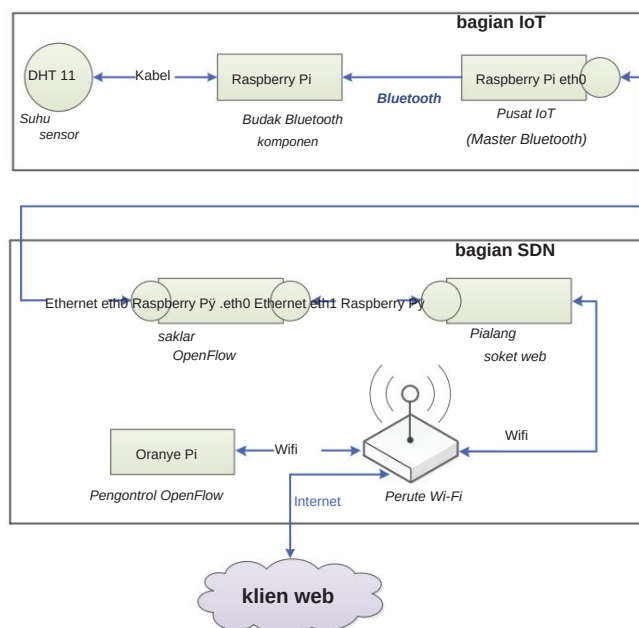
Pada bagian ini, kami memperkenalkan platform IoT berbasis SDN yang hemat energi untuk aplikasi yang sangat penting. Arsitektur platform terdiri dari bagian IoT dan SDN seperti yang ditunjukkan pada [Gambar 1](#). Bagian ini berkomunikasi melalui Bluetooth. Akses ke platform ini dipastikan melalui Internet menggunakan aplikasi Web.

Bagian IoT. Kami berasumsi bahwa layanan IoT penting dalam platform ini adalah transmisi data suhu dan kelembapan. Untuk tujuan ini, sensor suhu sederhana DHT11 digunakan [\[22\]](#).

DHT11 menyediakan pengukuran suhu sekitar dari 0°C hingga +50°C dengan akurasi $\pm 2^\circ\text{C}$. Sensor ini dihubungkan ke Raspberry Pi, yang bertindak sebagai pemancar dari sensor melalui Bluetooth Low Energy (BLE). Bersama dengan sensor, komponen Bluetooth di bagian IoT mengumpulkan data.

IoT Hub dirancang sebagai Raspberry Pi. IoT Hub mengumpulkan data dari beberapa komponen IoT, berkomunikasi melalui BLE. Dengan IoT Hub, data dapat diterima

sensor menggunakan algoritma yang efisien dan adaptif, yang menjadikan sistem produktif dan hemat energi [23].



Gambar 1: Arsitektur platform IoT berbasis SDN

bagian SDN. Jaringan SDN harus dapat mengkonfigurasi perangkat jaringan secara terprogram sesuai idenya. Pada gilirannya, jaringan ini harus terdiri dari setidaknya satu switch, dua host yang terhubung melalui switch ini, dan sebuah pengontrol yang memungkinkan jaringan untuk dikelola. Bagian SDN dalam arsitektur ini terdiri dari 3 komponen utama: saklar Openflow, broker WebSockets, dan pengontrol Openflow.

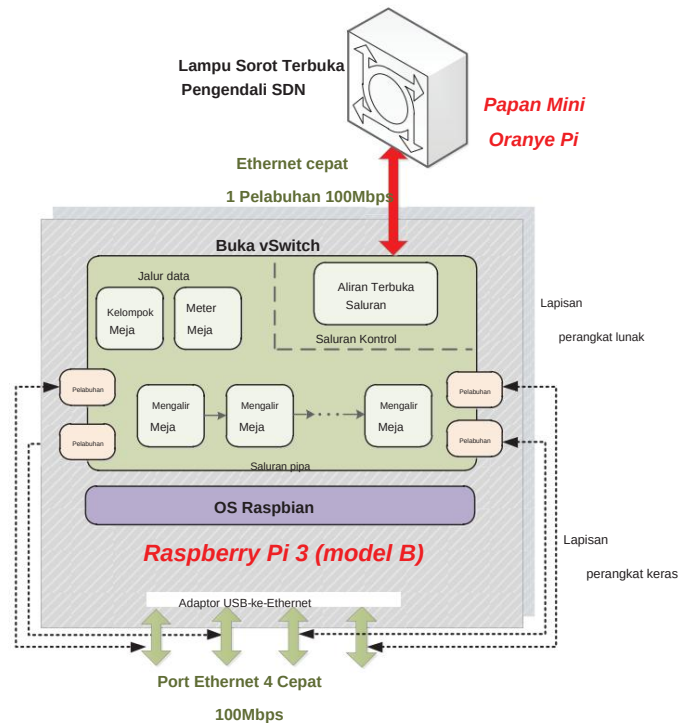
Kami mengembangkan sakelar SDN 5-port dengan perangkat Raspberry Pi 3 dan perangkat lunak Open vSwitch (OvS). Karena komputer papan tunggal Raspberry Pi di papannya hanya memiliki satu port Ethernet, penggunaan adaptor USB ke Ethernet pada papan ini dapat membuat hingga 5 port Ethernet.

Karena komputer papan tunggal Raspberry Pi pada dasarnya bukan saklar SDN, dengan menggunakan lingkungan tervirtualisasi perangkat keras, dimungkinkan untuk menginstal dan mengkonfigurasi saklar virtual. Implementasi paling populer dengan dukungan open source dan OpenFlow adalah OvS [24]. Sakelar OpenFlow berbasis Raspberry Pi bertindak sebagai sakelar virtual berbasis OvS. OvS adalah saklar virtual multilapis berkualitas produksi yang dilisensikan di bawah lisensi sumber terbuka Apache 2.0. Tujuan utama dari OvS adalah untuk menyediakan tumpukan peralihan untuk lingkungan tervirtualisasi perangkat keras untuk mendukung berbagai protokol dan standar, yang digunakan dalam jaringan komputer [25-27].

2.2 Uji Bukti Konsep untuk Platform IoT Berbasis SDN

Sakelar dan pengontrol SDN berbiaya rendah dan fleksibel berdasarkan komputer papan tunggal (Raspberry Pi 3 dan Orange Pi Prime) digambarkan pada Gambar 2.

OvS adalah implementasi perangkat lunak dari saklar jaringan multi-lapis virtual yang dirancang untuk memberikan otomatisasi jaringan yang efektif melalui ekstensi perangkat lunak.



Gambar 2: Sakelar dan pengontrol SDN berbiaya rendah dan fleksibel berdasarkan komputer papan tunggal (Raspberry Pi 3 dan jeruk Pi Prime)

OvS menyediakan konsep jembatan. Ini berarti satu saklar virtual dapat berisi banyak saklar virtual jembatan, yang pada gilirannya dapat menggunakan tabel routing yang berbeda atau port yang berbeda.

Untuk kenyamanan dan komunikasi real-time yang cepat antar layanan (bukan dalam mode permintaan-respons), protokol WebSocket dipilih. WebSocket adalah protokol yang dirancang untuk pertukaran informasi waktu nyata antara browser dan server web. Ini menyediakan saluran komunikasi dupleks penuh dua arah melalui satu soket TCP. WebSocket dirancang untuk diimplementasikan di browser Web dan server Web, tetapi juga dapat digunakan oleh aplikasi server-klien apa pun.

Antarmuka perangkat lunak aplikasi WebSocket telah distandarisasi oleh W3C (World Wide Web Consortium) dan WebSocket juga distandarisasi oleh IETF sebagai RFC 6455 [28].

Meskipun WebSocket terutama dirancang untuk komunikasi real-time antara klien Web dan layanan Web, WebSocket juga berguna untuk layanan. Kami berencana menggunakan protokol ini untuk komunikasi antara server Web dan broker.

Oleh karena itu, kami telah menerapkan logika berikut dalam bentuk langkah-langkah: • Broker menjalankan server WebSocket. • Layanan

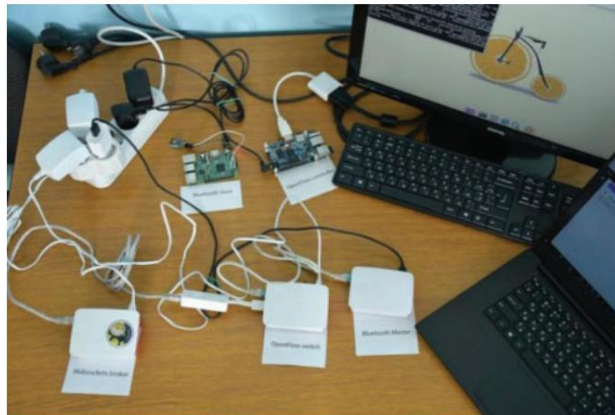
yang tertarik terhubung sebagai klien ke server WebSocket (klien Web dan pusat IoT).

• Setelah koneksi berhasil, klien web mengirimkan permintaan ke server WebSocket. • Pada gilirannya, broker (server WebSocket) mengirimkan permintaan ke hub IoT. • IoT Hub, pada gilirannya, membuat permintaan untuk komponen Bluetooth yang sudah terhubung perangkat.

- Saat komponen Bluetooth menerima permintaan tersebut, komponen tersebut meminta data dari sensor dan mengirimkannya kembali. Setiap perangkat, pada gilirannya, meneruskannya lebih jauh ke klien Web.

Setelah data dialihkan oleh tabel perutean ke port yang sesuai, data tersebut tiba di Websocket broker. Broker ini menggunakan protokol Websocket dengan nama yang sama dan merupakan servernya, yang menyediakan komunikasi real-time antara sensor akhir dan klien web.

Kami menggunakan papan Orange Pi Prime yang memiliki empat inti prosesor dan dua gigabyte RAM, yang seharusnya cukup untuk pengontrol SDN sederhana. Untuk mengimplementasikan platform IoT berbasis SDN, pengontrol Floodlight dipilih karena komponennya lebih sedikit, sehingga papan Orange Pi Prime akan mengatasi tugas tersebut dengan lebih baik. Floodlight menunjukkan dalam persyaratannya kebutuhan akan sistem operasi Ubuntu atau Debian. Sistem operasi Ubuntu untuk Orange Pi Prime dipilih karena popularitasnya. Testbed platform IoT berbasis SDN diberikan pada [Gambar 3](#).



Gambar 3: Bukti uji konsep untuk platform IoT berbasis SDN

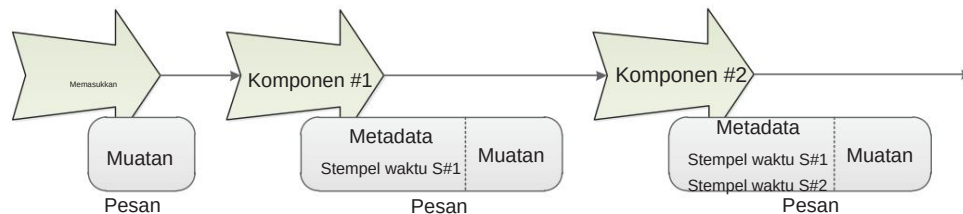
Dalam platform yang diusulkan, pengontrol SDN merupakan titik kontrol untuk memantau jaringan, mengendalikan aliran data, serta penyeimbangan beban, penskalaan otomatis, dan manfaat SDN lainnya. Pengontrol ini tidak mengalami kegagalan tunggal, yang berarti jaringan dapat terus beroperasi dalam mode normal. Pengontrol lampu sorot di antarmukanya memiliki dua cara mudah untuk mengelola jaringan-REST API dan GUI. GUI dibuat sebagai klien Web.

3 Metode Akurat Baru untuk Mengukur Delay End-to-End (E2E) di IoT Berbasis SDN Platform

3.1 Metode Pengukuran Delay pada Platform IoT Berbasis SDN

Dalam pekerjaan ini, sistem untuk memantau kualitas pengoperasian platform IoT berbasis SDN dikembangkan. Untuk menilai efisiensi operasinya dengan benar, beberapa kriteria kualitas sistem dipilih. Salah satunya adalah penundaan. Sampai saat ini, ini adalah salah satu kriteria utama kualitas sistem untuk layanan penting. Untuk tujuan ini, metode pengukuran penundaan E2E telah dikembangkan. Keunikan metode ini adalah memungkinkan penentuan penundaan pada setiap komponen platform yang diusulkan. Metode pengukuran penundaan membuat struktur data tertentu yang berisi metadata dan data yang berguna (payload) sebagai objek. Formatnya adalah JSON (JavaScript Object Notation), karena dalam format ini data dapat dibaca oleh manusia dan nyaman untuk diproses oleh komputer.

Setiap komponen menerima paket dan menambahkan stempel waktunya sendiri ke metadata, sehingga dengan membandingkan dua label satu sama lain, waktu tunda dapat ditentukan. Secara skematis, metode pengukuran penundaan ini diberikan pada Gambar 4. Untuk menghitung penundaan antara kedua modul, cukup dengan mengurangi cap waktu modul terakhir dari modul sebelumnya, dan selisihnya akan menjadi penundaan yang tepat.



Gambar 4: Prinsip kerja metode yang diusulkan

Metode ini mengharuskan jam pada semua komponen disinkronkan semaksimal mungkin untuk perhitungan yang akurat. Untuk ini kami menggunakan metode paling umum untuk menyinkronkan waktu sistem melalui jaringan di desktop atau server Linux adalah dengan menjalankan perintah *ntpdate* yang dapat mengatur waktu sistem dari server waktu NTP dengan kesalahan satu mikrodetik [29]. Jadi, ketika semua komponen berada dalam jaringan internal, termasuk broker, kesalahan pengukuran penundaan adalah satu mikrodetik. Jika broker berlokasi di jaringan eksternal, sulit untuk memastikan akurasi pengukuran penundaan yang tinggi, kesalahan dapat bervariasi dalam beberapa milidetik.

Metode ini memudahkan pengukuran penundaan antara komponen tertentu dan total penundaan E2E. Hal ini juga memungkinkan sistem untuk bereaksi dengan benar ketika penundaan meningkat secara dramatis atau mencapai ambang batas yang ditetapkan.

- N pertama = 10 pesan yang diterima dengan semua stempel waktu *stempel waktu*, dimana *stempel waktu* adalah variabel yang menunjukkan stempel waktu, komponen m (pengontrol SDN, saklar SDN, broker, sensor, dll.).
- Penundaan dihitung antara seluruh komponen jaringan menggunakan persamaan berikut

$$penundaan_{m2 \rightarrow m1} = stempel\ waktu_2 - stempel\ waktu_1 \quad (1)$$

- Nilai ambang batas untuk setiap penundaan komponen dihitung dengan cara

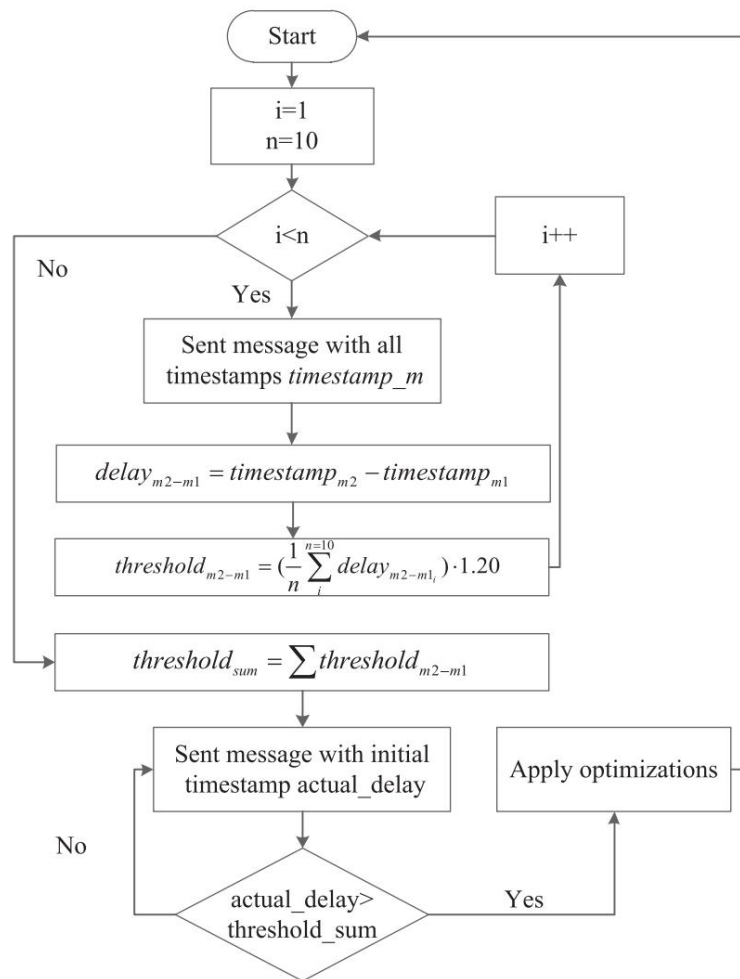
$$ambang\ batas_{m2 \rightarrow m1} = \frac{1}{N} \sum_{i=1}^{n=10} penundaan_{m2 \rightarrow m1 i} \cdot 1,20 \quad (2)$$

- dimana 1,20 adalah offset maksimum dari penundaan rata-rata.
- Total penundaan ambang batas dihitung dengan

$$jumlah\ ambang\ batas = ambang\ batas_{m2 \rightarrow m1} \quad (3)$$

- Selanjutnya, pesan hanya dikirim dengan stempel waktu awal untuk menentukan total penundaan.
- Jika total penundaan terlampaui, pesan berikut akan dikirim dengan stempel waktu dan penundaan tersebut dipantau pada setiap modul. Sistem akan secara otomatis menerima optimasi untuk menghilangkan penundaan yang tinggi untuk aplikasi yang sangat penting.
- Setelah menyelesaikan optimasi kita kembali ke langkah pertama.

Algoritma pengukuran penundaan pada platform IoT berbasis SDN diberikan pada [Gambar 5](#).



Gambar 5: Algoritma pengukuran penundaan pada platform IoT berbasis SDN

3.2 Hasil Eksperimen

Platform ini hanya memiliki satu switch yang menghubungkan dua host dan satu pengontrol SDN. Perangkat IoT diimplementasikan sebagai sensor suhu dan kelembaban sederhana. Sensor ini hanya mengirimkan dua parameter yaitu data pengukuran kelembapan dan suhu melalui Bluetooth. Klien Web diimplementasikan dalam dua pilihan: server Web dan klien Web berada di jaringan internal (localhost). Server Web ditempatkan dari jarak jauh di salah satu layanan hosting Web gratis, dan klien terhubung melalui jaringan seluler 4G (Heroku).

Salah satu tugas konsep IoT adalah mentransfer dan menyimpan data pengukuran dari berbagai sensor dengan kemungkinan pemrosesan dan penyimpanan lebih lanjut [30]. Pengukuran yang dikombinasikan dengan konsep IoT seperti “rumah pintar” dan “kota pintar” menjadi sangat penting. Di antara nilai yang paling sering diukur adalah parameter lingkungan dan parameter iklim dalam ruangan seperti suhu [31] dan kelembaban [32]. Perlu juga dicatat bahwa dua besaran fisis terakhir sangat penting dalam industri

[33–35]. Bahkan ada konferensi khusus yang didedikasikan untuk pengukurannya, seperti “TEMP-MEKO” dan “Suhu: pengukuran dan pengendaliannya dalam sains dan industri”. Oleh karena itu, karena pentingnya pengukuran suhu dan kelembaban, penulis memilih sensor suhu dan kelembaban untuk membuat testbed dalam pekerjaan ini.

Contoh sistem monitoring delay pada platform IoT berbasis SDN ketika web server dan web client berada pada jaringan internal-localhost digambarkan pada [Gambar 6](#). Pada sistem ini hanya hasil 8 percobaan transmisi data (pengukuran suhu) dan kelembaban) ditampilkan untuk kejelasan.

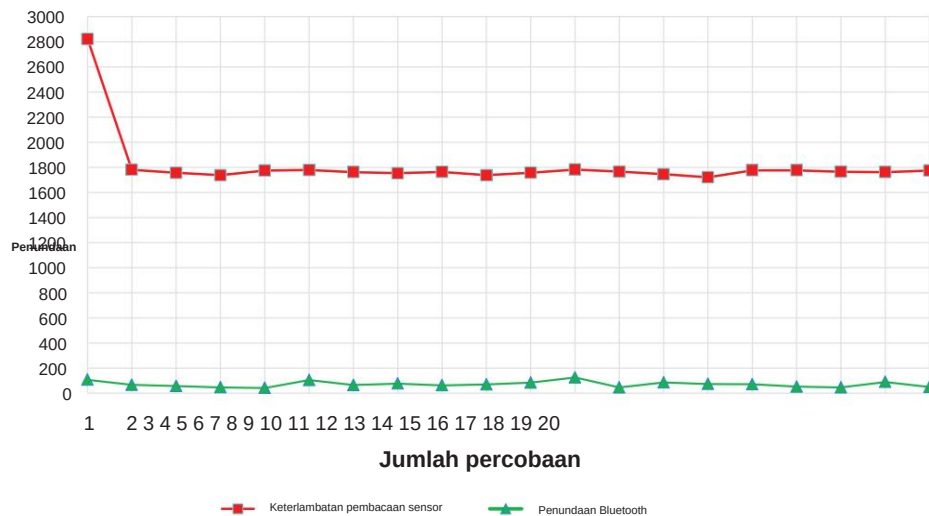
#	Temperature	Humidity	Sensor read latency (ms)	Bluetooth latency (ms)	SDN latency (ms)	WebSocket latency (ms)	Web latency (ms)	Total (ms)
1	21.0°C	28.0%	1761	136	4	15	148	2064
2	21.0°C	28.0%	1799	68	4	12	6	1889
3	22.0°C	28.0%	1782	44	4	14	6	1850
4	21.0°C	30.0%	1805	82	4	16	6	1913
5	21.0°C	34.0%	1800	22	5	11	7	1845
6	21.0°C	31.0%	1793	42	4	13	17	1869
7	21.0°C	28.0%	1775	102	4	15	9	1905
8	21.0°C	28.0%	1756	80	4	13	9	1862

Gambar 6: Sistem pemantauan penundaan pada platform IoT berbasis SDN (web server dan web client berada di jaringan internal-localhost)

20 percobaan transmisi data dilakukan untuk menilai rata-rata penundaan dan perilaku platform dengan lebih baik. Salah satu eksperimennya adalah membuat permintaan dari klien web dan mendapatkan respons dari klien web. Ada penundaan seperti: penundaan pembacaan sensor, penundaan Bluetooth, penundaan SDN, penundaan WebSocket, penundaan Web, penundaan total E2E.

Delay *baca Sensor* adalah penundaan pengiriman permintaan dan penerimaan data dari sensor dalam satuan milidetik. Dalam model ini, DTH11, sensor suhu dan kelembaban. Raspberry Pi menerima data ini melalui antarmuka GPIO. Nilai penundaan ini untuk 20 percobaan ditunjukkan pada [Gambar 7](#). Nilai rata-rata penundaan ini adalah 1814,4 milidetik. Deviasi standarnya adalah 238 milidetik. Terlihat dari grafik, nilai ini cukup konstan, kecuali untuk permintaan pertama. Nilai pengukuran penundaan yang pertama berbeda secara signifikan dengan nilai berikutnya karena pada awalnya, sebelum menyalakan prototipe, terdapat konfigurasi metode dan sinkronisasi waktu semua komponen. Oleh karena itu, ada penundaan tambahan. Selama percobaan kedua semua komponen disinkronkan dalam waktu. Setelah itu, pada percobaan selanjutnya tidak terjadi fluktuasi nilai penundaan yang tinggi. *Penundaan Bluetooth* adalah penundaan permintaan dan penerimaan data melalui antarmuka Bluetooth dalam milidetik. Raspberry Pi memiliki modul Bluetooth internal yang mendukung Bluetooth Low Energy (BLE). Standar ini beroperasi pada 2,4 GHz dan itu

kecepatan transfer maksimum adalah 1 Mbit/detik, namun nilai rata-rata dalam praktiknya adalah 0,27 Mbit/detik. Dalam platform ini, satu Raspberry Pi sebagai master Bluetooth mengirimkan permintaan ke budak Bluetooth yang koneksinya telah dibuat. Jumlah data dari sensor yang di request kurang lebih 200 byte. Nilai penundaan ini untuk 20 percobaan ditunjukkan pada [Gambar 9](#). Nilai rata-rata penundaan ini adalah 71,2 milidetik. Deviasi standarnya adalah 22,9 milidetik. Nilai ini tidak stabil dan sangat bervariasi.



Gambar 7: Pengukuran penundaan untuk bagian IoT (Pembacaan sensor dan penundaan Bluetooth)

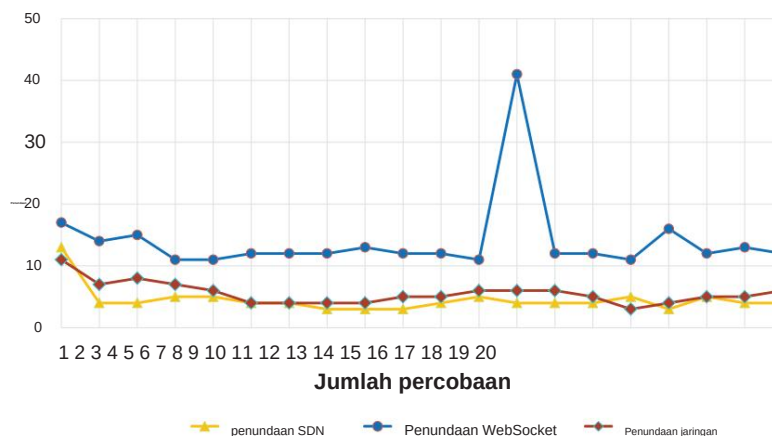
Penundaan SDN adalah penundaan yang terjadi saat melewati virtual Open vSwitch.

Raspberry Pi (IOT Hub) terhubung ke Raspberry Pi (broker Web) lain melalui Raspberry Pi perantara yang merupakan OvS virtual. Switch menghubungkan dua host melalui port Ethernet. Raspberry Pi mendukung 100 Base Ethernet, yaitu kecepatan transfer melalui port ini maksimal 100 Mbps. Switch ini hanya mempunyai dua nilai dalam tabel routingnya. Dua host berkomunikasi satu sama lain melalui WebSockets, dan jumlah data dalam satu pesan sekitar 1 kilobyte. Nilai penundaan ini untuk 20 percobaan ditunjukkan pada [Gambar 10](#). Nilai rata-rata penundaan ini adalah 4,5 milidetik. Deviasi standarnya adalah 2,1 milidetik. Nilai ini stabil dan tidak bervariasi banyak. Hanya pada permintaan pertama, penundaannya terasa lebih lama.

Penundaan WebSocket terjadi ketika kami mengirim permintaan dari server web ke broker dan menerima respons. Broker dan server web berkomunikasi melalui protokol WebSocket. Penundaan sudah diukur saat koneksi dibuat. Perlu diperhatikan bahwa dalam percobaan ini, server web berada di jaringan lokal. Volume data pesannya juga sekitar 1 kilobyte. Nilai penundaan ini selama 20 kali percobaan ditunjukkan pada [Gambar 10](#). Nilai rata-rata penundaan ini adalah 14,05 milidetik. Deviasi standarnya adalah 6,6 milidetik. Nilai ini cukup stabil dan sedikit bervariasi, namun terkadang bisa jauh lebih lama karena berbagai alasan.

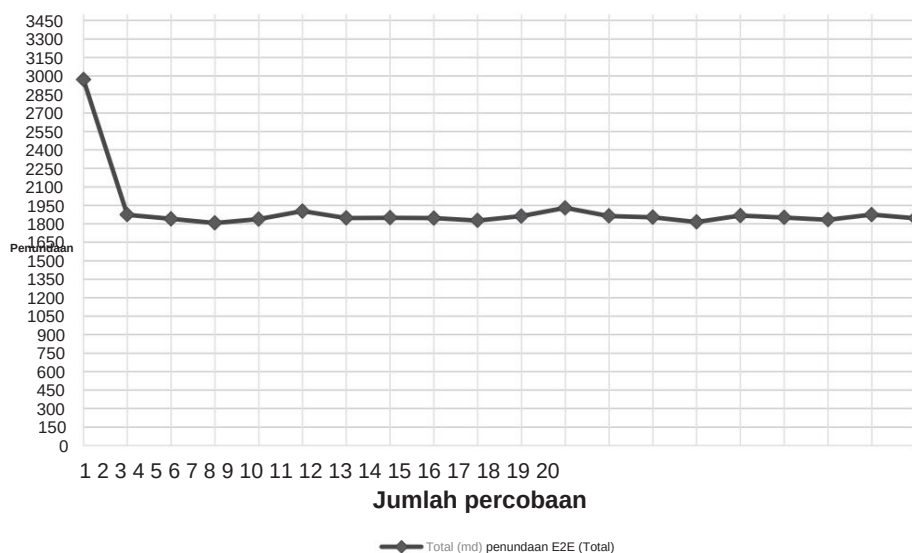
Penundaan Web adalah penundaan (diukur dalam milidetik) yang terjadi ketika klien web membuat permintaan dan menerima respons dari server web. Permintaan dibuat menggunakan protokol HTTP, dan responsnya berukuran 1 kilobyte. Perlu dicatat bahwa klien web dan server web berada di jaringan internal yang sama. Nilai penundaan ini untuk 20 percobaan ditampilkan

pada [Gambar 8](#). Nilai rata-rata penundaan ini adalah 5,55 milidetik. Deviasi standarnya adalah 1,8 milidetik. Nilai ini tidak berkelanjutan.



Gambar 8: Pengukuran penundaan bagian SDN (SDN, WebSocket, dan penundaan Web)

Jadi jika kita jumlahkan semua penundaan ini, untuk percobaan sebelumnya, kita mendapatkan plot berikut pada [Gambar. 9](#) dari total penundaan E2E, yaitu penundaan dari saat pengguna klien web mengklik tombol permintaan data sebelum menerimanya di tabel di klien web. Total penundaan rata-rata adalah 1909,6 milidetik dan standar deviasinya adalah 251,4 milidetik. Namun terlihat dari gambar, setelah permintaan pertama, nilai penundaannya cukup stabil.



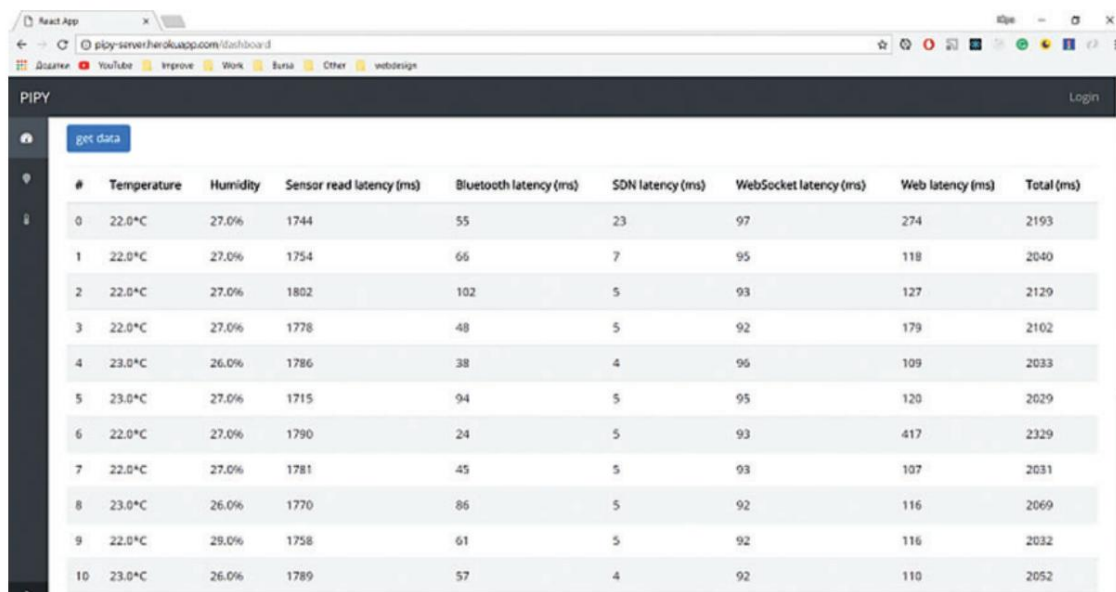
Gambar 9: Total penundaan E2E pada platform IoT berbasis SDN yang diusulkan

Terlihat dari plot, yang paling tinggi diantara semua delay adalah delay-delay baca sensor dalam membaca data dari sensor. Yang tertinggi kedua adalah penundaan Bluetooth—penundaan dalam pertukaran

pesan melalui protokol Bluetooth. Menurut bagian IoT, penundaannya adalah $71.2 + 1814.4 = 1885.6$ milidetik sedangkan penundaan bagian SDN hanya $1909.6 - 1885.6 = 24$ milidetik.

Namun pada percobaan ini web client dan web server berada pada jaringan internal.

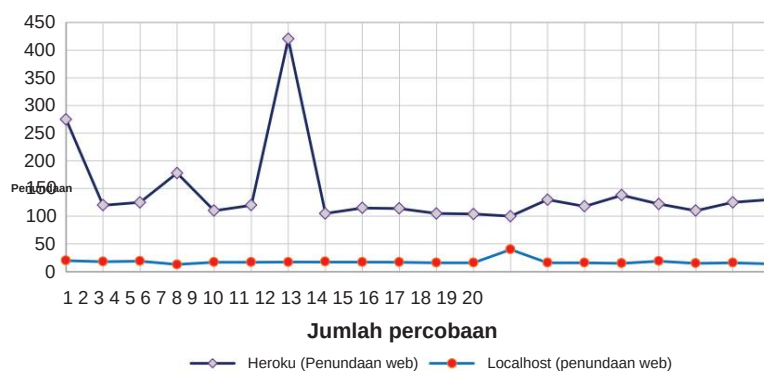
Klien sebenarnya akan mengakses server jarak jauh dari Internet. Hal ini akan mengakibatkan penundaan yang lebih tinggi. Oleh karena itu, untuk memeriksanya, server web dikerahkan pada web hosting jarak jauh Heroku dan klien terhubung melalui jaringan seluler 4G. [Gambar 10](#) menunjukkan sistem pemantauan penundaan untuk platform IoT berbasis SDN. Terlihat dari gambar, semua penundaan kurang lebih sama jika dibandingkan dengan jaringan internal (localhost), kecuali penundaan Web dan WebSocket.



#	Temperature	Humidity	Sensor read latency (ms)	Bluetooth latency (ms)	SDN latency (ms)	WebSocket latency (ms)	Web latency (ms)	Total (ms)
0	22.0°C	27.0%	1744	55	23	97	274	2193
1	22.0°C	27.0%	1754	66	7	95	118	2040
2	22.0°C	27.0%	1802	102	5	93	127	2129
3	22.0°C	27.0%	1778	48	5	92	179	2102
4	23.0°C	26.0%	1786	38	4	96	109	2033
5	23.0°C	27.0%	1715	94	5	95	120	2029
6	22.0°C	27.0%	1790	24	5	93	417	2329
7	22.0°C	27.0%	1781	45	5	93	107	2031
8	23.0°C	26.0%	1770	86	5	92	116	2069
9	22.0°C	29.0%	1758	61	5	92	116	2032
10	23.0°C	26.0%	1789	57	4	92	110	2052

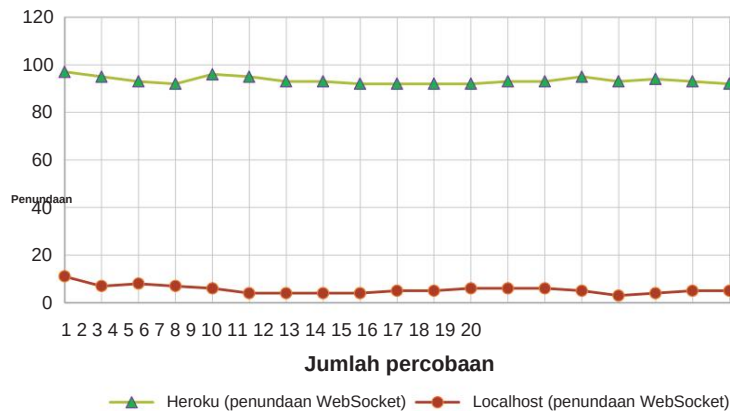
Gambar 10: Sistem pemantauan penundaan untuk platform IoT berbasis SDN (server web diterapkan pada hosting web jarak jauh Heroku dan klien terhubung melalui jaringan seluler 4G)

Perbandingan penundaan web (Heroku vs. Localhost) digambarkan pada [Gambar 11](#). Dalam hal ini, nilai rata-rata penundaan adalah 141,3 milidetik dan standar deviasinya adalah 73,3 milidetik.



Gambar 11: Perbandingan penundaan web (Heroku vs. Localhost)

Perbandingan penundaan WebSocket (Heroku vs. Localhost) digambarkan pada [Gambar 12](#).



Gambar 12: Perbandingan penundaan WebSocket (Heroku vs. Localhost)

Sebagai hasil dari pengujian platform IoT berbasis SDN, kami menemukan bahwa penggunaan teknologi SDN memastikan persyaratan yang diperlukan untuk penundaan transmisi data penting. Menurut metode yang diusulkan, untuk mengurangi penundaan di bagian IoT, perlu untuk memilih sensor suhu dan kelembaban dengan parameter teknis terbaik untuk membaca dan memproses informasi dengan cepat. Solusi ini akan meningkatkan parameter kualitas layanan secara signifikan.

Dalam karya ini, SDN diimplementasikan untuk Internet of Things berbasis komputer papan tunggal. Platform ini merupakan bukti konsep (PoC) dan fokus pada implementasi kinerja, kemungkinan adanya ide dan analisis sistem. Namun, solusi akhirnya bisa sangat beragam, sesuai dengan kebutuhan yang berbeda. Saat membuat solusi akhir, penting untuk mempertimbangkan berbagai parameter yang memungkinkan pengorganisasian arsitektur secara keseluruhan dan dalam memecahkan masalah spesifik dengan lebih baik dan lebih cerdas. Saat merancang jaringan yang ditentukan perangkat lunak untuk IoT, seseorang harus memperhatikan jenis data yang direncanakan untuk dikirimkan.

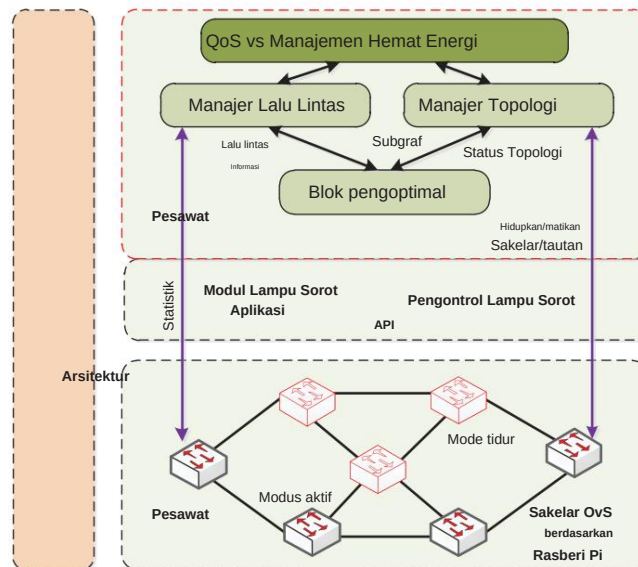
Klasifikasi data real-time dan unreal-time serta volumenya sangatlah penting. Jika sistem menyediakan transmisi data real-time, maka perlu menjaga QoS yang disediakan oleh jaringan komunikasi. Bagian IoT diperlukan untuk memastikan protokol transmisi data kabel atau nirkabel berkecepatan tinggi. Dalam kasus kami, sensor suhu DHT11 perlu diganti dengan sensor yang lebih cepat, yang akan mengurangi penundaan E2E. SDN menawarkan solusi terbaik dan nyaman untuk memastikan QoS, justru karena kemampuannya mengendalikan jaringan secara terprogram. Seorang administrator dapat secara dinamis dan fleksibel menentukan kebijakan jaringan untuk mengalokasikan sumber daya jaringan ke aliran prioritas yang berbeda. Misalnya, administrator dapat memilih throughput jaringan untuk aliran prioritas, waktu tunda jalur, atau kriteria lain sebagai jalur komunikasi optimal untuk aliran data tertentu. Itulah sebabnya di bagian pekerjaan selanjutnya, kami menawarkan pendekatan untuk perutean optimal aliran data dari berbagai kelas IoT berdasarkan kriteria kualitas layanan untuk jaringan skala besar (dengan sejumlah besar sakelar SDN).

4 Pendekatan Efisiensi Energi pada SDN 4.1

Pendekatan Efisiensi Energi Proporsional pada Platform IoT Berbasis SDN

Karena biaya energi berkontribusi signifikan terhadap total biaya jaringan, efisiensi energi merupakan persyaratan penting ketika merancang mekanisme jaringan modern, terutama dalam infrastruktur IoT. Namun, mengembangkan solusi hemat energi adalah tugas yang kompleks, karena terdapat trade-off antara efisiensi energi dan kinerja jaringan [36]. SDN memastikan kemampuan program elemen jaringan dengan memisahkan bidang kendali dan bidang penerusan. Karena perangkat SDN dapat diprogram dan dikelola, sakelar jaringan yang diusulkan berdasarkan satu papan dan tautan dapat memasuki kondisi daya rendah dalam mode tidur, dan arsitektur perangkat dapat dirancang untuk memiliki beban aktual berdasarkan konsumsi energi yang proporsional.

Kami mengusulkan pendekatan untuk merancang struktur SDN yang hemat energi sehubungan dengan beban jaringan yang dihasilkan oleh perangkat IoT. Pendekatan ini mendukung kompromi antara efisiensi energi dan kinerja. Arsitektur SDN hemat energi untuk komunikasi IoT digambarkan pada Gambar 13.



Gambar 13: Arsitektur SDN efisiensi energi untuk komunikasi IoT (garis putus-putus merah adalah blok program yang kami usulkan)

Prinsip pendekatan yang diusulkan adalah sebagai berikut. Dengan dukungan pengontrol SDN, struktur jaringan dipantau dan dikelola secara terpusat. Blok pengoptimal dikembangkan pada pengontrol SDN untuk mengumpulkan statistik dari sakelar OvS tentang topologi jaringan dan beban sakelar. Berdasarkan statistik terkini tentang keadaan jaringan, keputusan adaptif dibuat untuk membangun topologi jaringan yang optimal sehubungan dengan kriteria QoS dan konsumsi energi. Dalam kondisi beban jaringan rendah yang mendukung pendekatan ini, pengontrol SDN, berdasarkan data yang diterima dari blok pengoptimal jaringan, akan memutuskan untuk membuat grafik (topologi) dengan jumlah tautan dan sakelar aktif yang lebih sedikit, sedangkan beban jaringan tinggi akan meningkatkan jumlah sakelar aktif dan hubungan di antara mereka dalam grafik.

Berdasarkan model perutean yang dijelaskan di atas, penyesuaian biaya jalur dapat dilakukan dengan menggunakan korektif

bobot untuk memperhitungkan seberapa besar penundaan atau kehilangan paket yang penting untuk aliran IoT tertentu. Dengan cara ini, kita dapat menyesuaikan parameter ini agar sesuai dengan persyaratan kualitas layanan dari setiap aliran. Oleh karena itu, penyeimbangan beban antar tautan dapat dilakukan sehingga lebih sedikit sakelar yang terlibat dan sakelar yang mengganggu dapat dialihkan ke mode tidur untuk meningkatkan efisiensi energi jaringan secara keseluruhan. Ketika beban dari perangkat IoT bertambah dan QoS menurun, keputusan otomatis dibuat untuk mengaktifkan sakelar tidur dan mengalihkannya ke mode aktif. Beban didistribusikan ulang menggunakan model matematika berikut.

Kami menyajikan bagian SDN pada platform IoT dalam bentuk grafik berbobot $G = (N, M)$, di mana N adalah jumlah sakelar OvS berdasarkan Raspberry Pi dan $N_i \in N$ adalah sakelar ke- i OvS. Kami juga menyajikan saluran $m_{ij} \in M$ antara sakelar N_i dan N_j . Parameter $B_{i,j}$ merupakan throughput yang menghubungkan saklar OvS N_i dan N_j . Biarkan variabel biner V_i dan L_{ij} menunjukkan keadaan saklar N_i dan saluran m_{ij} seperti

$$V_i = \begin{cases} 1, & \text{jika OvS } N_i \text{ aktif} \\ 0, & \text{sebaliknya} \end{cases}$$

$$L_{ij} = \begin{cases} 1, & \text{jika saluran } m_{ij} \text{ aktif} \\ 0, & \text{sebaliknya} \end{cases}$$

P_i adalah konsumsi energi OvS N_i dan C_{ij} adalah konsumsi energi saluran m_{ij} yang diukur dalam kilowatt atau kilojoule per jam.

Lalu lintas jaringan diwakili oleh sekumpulan aliran F , di mana $f \in F$ didefinisikan sebagai $f = (sr, ds, \gamma f)$, di mana sr dan ds adalah saklar OvS sumber dan tujuan, dan γf adalah laju aliran, diukur dalam byte per detik.

$$f_{ij} = \begin{cases} 1, & \text{jika aliran } f \text{ melewati tepi } e_{ij} \\ 0, & \text{sebaliknya} \end{cases}$$

$$L_{ij} = \begin{cases} 1, & \gamma_{\min} \leq \gamma f \leq \gamma_{\max} \\ 0, & \text{sebaliknya} \end{cases}$$

dimana $\gamma_{\min}$ dan $\gamma_{\max}$ adalah beban saluran minimum dan maksimum untuk menjaga trade-off antara kinerja dan efisiensi energi.

Fungsi multiobjektif ([Persamaan \(1\)](#)) meminimalkan jumlah konsumsi energi sakelar dan saluran. Ekspresi matematis pertama dari fungsi tujuan, jumlah $f_{ij} \cdot C_{ij}$, mengacu pada total konsumsi energi semua aliran yang menggunakan saluran m_{ij} . Ekspresi matematika kedua adalah jumlah total konsumsi energi semua switch OvS aktif di jaringan SDN. Fungsi tujuan secara kolektif meminimalkan jumlah yang terkena batasan yang dijelaskan di bawah.

$$\text{menit } \gamma \quad L_{ij} \cdot C_{ij} + \quad V_i \cdot P_i \quad \gamma \quad (4)$$

$$\text{tunduk pada} \quad f_{ij} \cdot \gamma f \leq B_{ij}, \quad \gamma_{m_{ij}} \quad \gamma f \quad (5)$$

$$f_{ij} = \sum_{f} f_{ij}, N_i \& N_j = s_r, N_i \& N_j = d_s \quad (6)$$

$$f_{kj} = f_{iq}, N_k = s_r, N_q = d_s, \sum_{k} f_{kj}, \sum_{i} f_{miq} \quad (7)$$

$$f_{ij} \in V_j \text{ dan } f_{ji} \in V_j, \sum_{j} N_j \in N \quad (8)$$

$$f_{ij} \in V_j \sum_{j} N_j \in N \quad (9)$$

$$V_i \in \left[\sum_{f} f_{ij} + f_{ji} \right], \sum_{i} N_i \in N \quad (10)$$

$$L_{ij} \in V_i, L_{ij} \in V_j \sum_{i} N_i \quad (11)$$

Dalam (Persamaan (4)), dinyatakan bahwa laju aliran total antara kedua sakelar tidak boleh melebihi throughput saluran Bi,j. Persamaan. (5) dan (6) keadaan konservasi aliran, yang menyatakan tidak ada aliran yang tercipta atau hilang dalam jaringan. Kendala dalam Persamaan. (6) menyatakan bahwa jumlah arus yang masuk dan keluar dari saklar-saklar yang bukan merupakan tujuan maupun sumber aliran harus genap. Kendala untuk Persamaan. (4) memastikan bahwa aliran yang berasal dari sakelar sumber harus meninggalkan jaringan di sakelar tujuan. Kendala untuk Persamaan. (8)–(10) mendukung korelasi antara sakelar dan saluran menggunakan variabel status sakelar dan variabel saluran aliran. Batasan 8 dan 9 menyatakan bahwa tidak boleh ada aliran yang menggunakan saluran yang terhubung ke sakelar yang tidak aktif, sedangkan batasan 10 menyatakan bahwa jika tidak ada aliran yang melewati saluran yang terhubung ke sakelar tertentu, maka sakelar tersebut akan mati. Kendala dalam Persamaan. (11) menyatakan bahwa saluran yang terhubung ke saklar non-aktif harus dinonaktifkan. Mengingat formulasi dan parameter model, hasil pengoptimal adalah daftar link dan sakelar aktif untuk setiap aliran $f \in F$.

Oleh karena itu, pendekatan kami memberikan penghematan yang signifikan dalam konsumsi energi secara keseluruhan jaringan sambil mencapai kinerja optimal untuk memenuhi persyaratan QoS yang diperlukan.

4.2 Hasil Eksperimen

Raspberry Pi 3 dapat ditenagai dengan catu daya sebesar 5 Volt. Untuk penggunaan normal, Raspberry Pi hanya menggunakan $P = 2,25$ W dan konsumsi energi $E = 8,1$ kilojoule per jam [kJ/h]. Misalnya, jika jaringan kami terdiri dari 7 sakelar OvS berbasis Raspberry Pi, konsumsi daya dalam mode tidur dapat diabaikan. Konsumsi energi dalam mode tidur sangat kecil.

Total konsumsi energi jaringan per hari dihitung dengan Persamaan. (12):

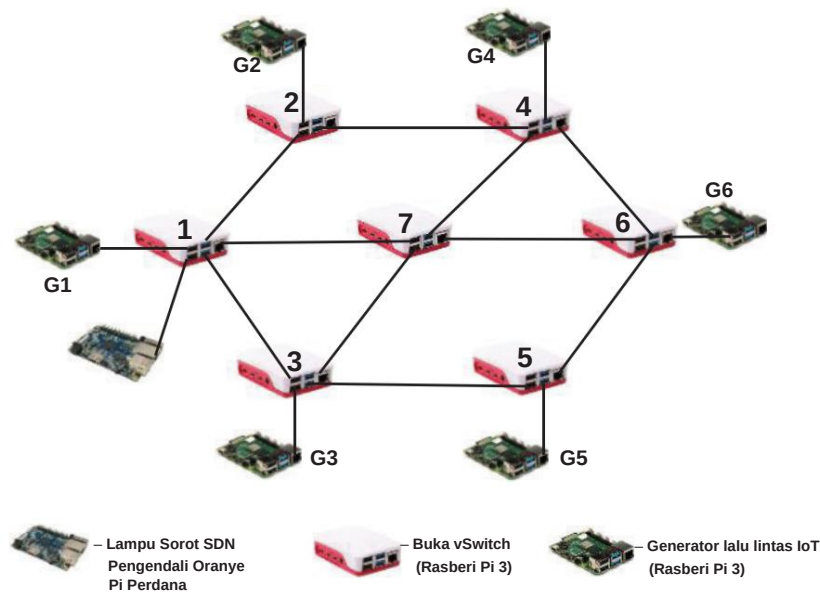
$$Jaringan E = \sum_{i=1}^{24} (E \cdot n)_i \text{ [kJ/jam]}, \quad (12)$$

dimana n-jumlah sakelar SDN aktif berdasarkan Raspberry Pi, E—konsumsi energi dari satu sakelar SDN.

Penghematan energi dihitung dengan [Persamaan. \(13\)](#).

$$\text{Penghematan keenergi} = 1 - \frac{\text{Jaringan dengan solusi kami}}{\text{Enetworktanpa solusi kami}} \cdot 100 \quad (13)$$

Untuk pengujian eksperimental kami, kami mempertimbangkan topologi jaringan berdasarkan komputer papan tunggal. Topologi ini terdiri dari 7 Open vSwitches, 1 pengontrol Floodlight SDN, dan 6 lalu lintas IoT generator (G1, G2, G3, G4, G5, dan G6). Testbed SDN eksperimental digambarkan pada [Gambar 14](#).



Gambar 14: Testbed eksperimental SDN berdasarkan komputer papan tunggal

Kami menghasilkan beban berbeda selama 24 jam di jaringan. Untuk mencapai beban rendah pada jaringan, kami tidak menghasilkan beban dengan beberapa generator G1, G2, G3, G4, G5 dan G6. Kami memperkirakan konsumsi energi dengan dan tanpa solusi efisiensi energi kami untuk layanan a beban tertentu. Jadi, rata-rata beban jaringan per hari digambarkan pada [Gambar 15](#). Jumlah yang aktif Switch SDN berbasis Raspberry Pi per hari digambarkan pada [Gambar 16](#). Konsumsi energi perbandingan dalam kilojoule per hari ditunjukkan pada [Gambar 17](#).

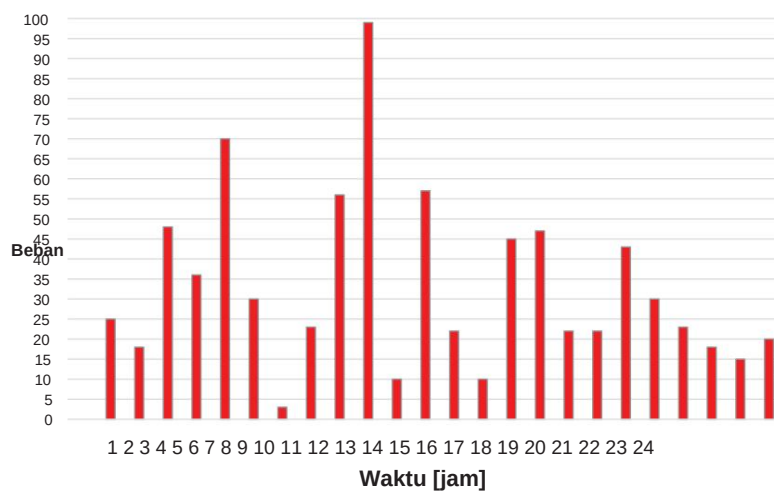
Hasil eksperimen menunjukkan bahwa total konsumsi energi jaringan tanpa solusi kami adalah *Enetworktanpa solusi kami* = 1361 [kJ/h] dan dengan solusi kami (pendekatan efisiensi energi)

$$\text{Jaringan dengan solusi kami} = 632 \text{ [kJ/jam]}. \text{ Kemudian penghematan energi setelah menyadari pendekatan kami adalah}$$

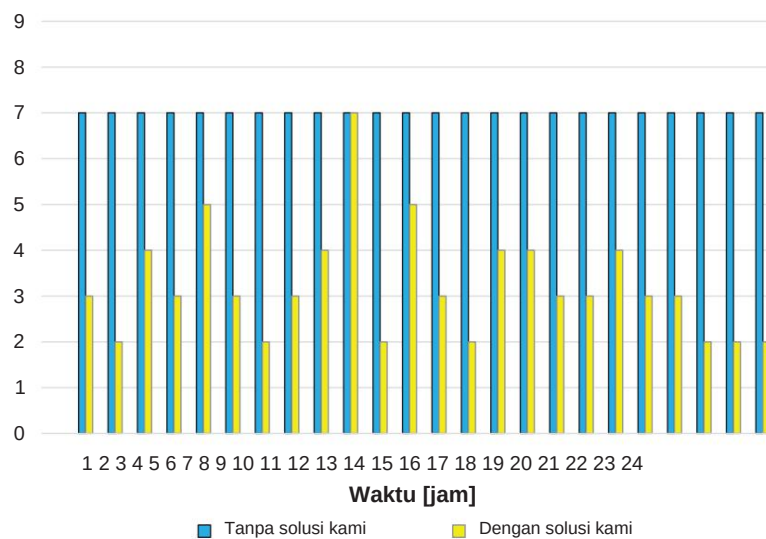
$$\text{Penghematan keenergi} = \left(1 - \frac{632}{1361}\right) \cdot 100\% = 53,56\%.$$

Eksperimen yang dilakukan pada topologi SDN nyata berdasarkan 7 komputer papan tunggal Rasp-berry Pi 3, switch, dan lalu lintas IoT menunjukkan bahwa pendekatan kami tidak hanya menghemat hingga 53,56% dari energi pada volume lalu lintas rendah, tetapi juga memastikan QoS untuk aplikasi IoT.

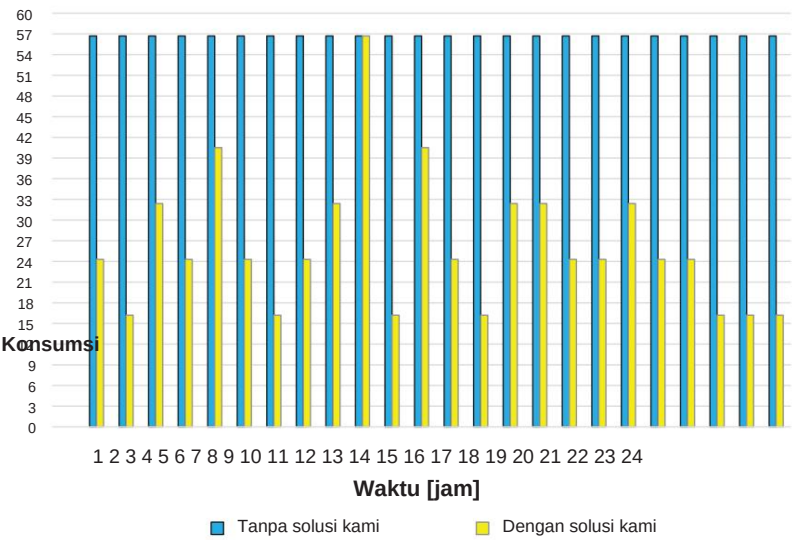
Ringkasan pendekatan yang dibahas untuk mengurangi konsumsi energi dan meningkatkan kualitas layanan dalam jaringan modern ditunjukkan pada [Tab. 1](#). Pekerjaan kami berbeda dari pekerjaan sebelumnya dalam hal metode pengukuran penundaan, alokasi sumber daya, dan implementasi prototipe.



Gambar 15: Rata-rata beban jaringan per hari



Gambar 16: Jumlah switch SDN aktif berdasarkan Raspberry Pi per hari



Gambar 17: Perbandingan konsumsi energi dalam kilojoule per jam selama sehari

Tabel 1: Perbandingan karya-karya terbaru

Deskripsi Referensi	Bandingkan dengan pekerjaan kita
[37] Pendekatan ini didasarkan pada manajemen sumber daya server untuk mengurangi kemacetan dan energi dalam jaringan VoIP. Selain itu, penulis menggunakan switch OpenFlow daripada switch tradisional dalam infrastruktur jaringan. Hasilnya, hal ini secara efektif mencegah kemacetan, dan jumlah peralatan aktif dalam jaringan diminimalkan	Berbeda dengan pekerjaan kami, penulis di [37] mengusulkan untuk menghemat energi dengan mematikan server tergantung pada bebannya menggunakan sakelar OvS. Kami mengusulkan untuk melakukan pemantauan terpusat dan pengelolaan struktur jaringan menggunakan pengontrol SDN logika canggih untuk mematikan sakelar, termasuk yang dibangun pada platform Raspberry. Untuk tujuan ini, kami telah mengembangkan dan berhasil mengimplementasikan pengoptimal blok di pengontrol.

(Lanjutan)

Tabel 1: Lanjutan

Deskripsi Referensi [38]	Perbandingan dengan pekerjaan
Strategi yang diusulkan oleh penulis adalah menggabungkan perutean terpendek berdasarkan prioritas dan perencanaan aliran eksklusif dalam jaringan pusat data yang ditentukan perangkat lunak. Ini memiliki peningkatan efisiensi energi yang jelas untuk mentransfer file besar di jaringan BCube. Dalam hal ini, perutean EXR yang diprioritaskan kira-kira 5%–35% lebih hemat energi dibandingkan strategi umum lainnya.	kami Berbeda dengan pekerjaan routing sebelumnya yang berfokus pada masalah minimalisasi energi di jaringan pusat data, kami bertujuan untuk mengoptimalkan konsumsi energi di Platform IoT Berbasis SDN menggunakan Raspberry Pi 3. Eksperimen yang dilakukan pada topologi SDN/ IoT nyata berdasarkan komputer papan tunggal menunjukkan bahwa pendekatan kami tidak hanya menghemat hingga 53,56% energi pada intensitas lalu lintas rendah, namun juga memberikan jaminan QoS untuk aplikasi IoT berdasarkan sistem pemantauan penundaan E2E baru. . Penulis artikel [38] hanya membahas prioritas arus untuk menemukan jalur transmisi yang lebih hemat energi. Hasil yang diperoleh juga tidak diterapkan dalam praktik, sehingga menimbulkan keraguan mengenai penerapan dan manfaatnya.
[39] Pendekatan yang diusulkan oleh penulis secara efisien mengurangi energi infrastruktur sekaligus membangun rute dinamis dengan menggabungkan aliran di seluruh sumber daya jaringan yang diaktifkan. Dengan strategi kesadaran energi ini mendukung perlunya melakukan kompromi berdasarkan jenis aliran antara jaminan layanan, optimalisasi alokasi sumber daya, dan pengurangan konsumsi energi untuk meningkatkan kinerja jaringan secara keseluruhan.	Berbeda dengan penelitian yang ada [39], di mana penulis hanya bekerja dengan penundaan E2E, kami mengevaluasi penundaan yang ditimbulkan oleh setiap komponen infrastruktur IoT, mulai dari sensor, inti SDN, broker IoT, dan pelanggan layanan IoT. Pendekatan ini akan mengidentifikasi hambatan dalam infrastruktur yang menyebabkan penurunan parameter QoS dan secara otomatis memutuskan apakah akan menghilangkan hambatan tersebut atau secara optimal membangun topologi untuk mengurangi konsumsi energi jaringan.

5. Kesimpulan

Harga penerapan yang besar dan persaingan pasar yang tinggi menyebabkan penyedia layanan IoT seringkali tidak mendapatkan minat yang memadai dari investasi mereka. Untuk mengatasi masalah ini, kami menawarkan solusi baru bagi usaha kecil dan menengah yang memungkinkan mereka memasuki pasar IoT dengan investasi yang jauh lebih rendah. Solusi yang diusulkan memungkinkan penggelaran jaringan yang dapat diprogram sesuai permintaan untuk berbagai aplikasi IoT, termasuk layanan yang sangat penting. Kami mengembangkan platform bukti konsep yang menghubungkan berbagai perangkat akhir IoT dan menyediakan mekanisme yang fleksibel dan terukur untuk kontrol QoS E2E untuk aplikasi yang sangat penting. Platform yang diusulkan terdiri dari seperangkat komputer papan tunggal yang hemat energi dan biaya serta menggunakan fitur unik SDN, seperti sifat berbasis aliran, dan fleksibilitas jaringan untuk memenuhi QoS.

persyaratan setiap aliran IoT di jaringan. Keuntungan utama dari prototipe jaringan SDN yang dikembangkan adalah biaya rendah dan ketersediaan implementasi, yang penting untuk pelatihan spesialis di bidang jaringan yang ditentukan perangkat lunak dalam proses tujuan pendidikan, pelatihan dan penelitian.

Dalam makalah ini, pertama-tama kami mempresentasikan dan menerapkan metode untuk mengukur penundaan yang diperkenalkan oleh setiap komponen infrastruktur IoT, mulai dari sensor, inti SDN, broker IoT, hingga pelanggan IoT. Kami juga mempresentasikan pendekatan hemat energi untuk platform IoT berbasis SDN yang sebanding dengan intensitas lalu lintas IoT. Pendekatan ini menghemat energi dengan secara dinamis menyalakan sejumlah minimum perangkat jaringan untuk mengirimkan lalu lintas sehubungan dengan beban jaringan daripada terus-menerus membiarkan semua saklar tetap menyala, seperti pada jaringan saat ini.

Keterbatasan pekerjaan ini adalah throughput saluran komunikasi pada level switch yang dirancang adalah 100 Mbps, karena switch dirancang berdasarkan board Raspberry Pi 3 menggunakan Open vSwitch (OvS). Raspberry Pi hanya mendukung 100 Base Ethernet. Oleh karena itu kedepannya kami akan menguji switch jaringan IoT backbone ZODIAC FX yang memiliki throughput 1000 Mbps.

Pekerjaan kami di masa depan adalah mengembangkan model perutean QoE (Kualitas Pengalaman) yang hemat energi untuk jaringan berbasis tujuan yang ditentukan perangkat lunak di masa depan. Kebaruan model ini adalah bahwa metrik rute berorientasi QoE adaptif akan digunakan untuk memilih jalur transmisi optimal, berdasarkan model matematika korelasi QoS/QoE yang dikembangkan sehubungan dengan parameter fungsional pemanfaatan node jaringan. Sebagai hasil dari implementasi perangkat lunak pada pengontrol SDN/IBN, akan dimungkinkan untuk mempertahankan kompromi antara kualitas pengguna layanan yang berorientasi pada intensitas yang diinginkan, pemanfaatan jaringan, dan efisiensi energi dengan menempatkan node yang menganggur ke mode hemat energi. Secara khusus, kami akan menerapkan pendekatan ini pada switch ZODIAC FX/GX SDN asli, yang terbuka untuk peningkatan logika manajemen jaringan. Kami sudah memiliki beberapa pencapaian sukses dari prototipe ini, yang akan segera kami ulas dalam karya kami selanjutnya.

Pernyataan Pendanaan: Penelitian ini didukung oleh Proyek Ukraina No. 0120U102201 "Pengembangan metode dan sarana perangkat lunak-perangkat keras terpadu untuk penerapan jaringan informasi dan komunikasi multiguna berbasis tujuan yang hemat energi".

Konflik Kepentingan: Para penulis menyatakan bahwa mereka tidak memiliki konflik kepentingan untuk dilaporkan mengenai penelitian ini.

Referensi

- [1] PKR Maddikunta, TR Gadekallu, R. Kaluri, G. Srivastava, RM Parizi *dkk.*, "Komunikasi ramah lingkungan dalam jaringan IoT menggunakan algoritma optimasi hibrid," *Computer Communications*, vol. 159, hlm.97–107, 2020.
- [2] PKR Maddikunta, G. Srivastava, TR Gadekallu, N. Deepa dan P. Boopathy, "Model prediktif untuk masa pakai baterai di jaringan IoT," *IET Intelligent Transport Systems*, vol. 14, tidak. 11, hlm.1388–1395, 2020.
- [3] Q. Pham, S. Mirjalili, N. Kumar, M. Alazab dan W. Hwang, "Algoritma pengoptimalan paus dengan aplikasi untuk alokasi sumber daya di jaringan nirkabel," *IEEE Transactions on Vehicular Technology*, vol. 69, tidak. 4, hal.4285–4297, 2020.
- [4] PKR Maddikunta, G. Srivastava, TR Gadekallu, N. Deepa dan P. Boopathy, "Model prediktif untuk masa pakai baterai di jaringan IoT," *IET Intelligent Transport Systems*, vol. 14, tidak. 11, hlm.1388–1395, 2020.
- [5] W. Anwaar dan MA Shah, "Komputasi hemat energi: Perbandingan raspberry pi dengan perangkat modern," *Jurnal Internasional Komputer dan Teknologi Informasi*, vol. 4, tidak. 2, hal.410–413, 2015.
- [6] H. Kim, J. Kim dan Y. Ko, "Mengembangkan pengujian aliran terbuka yang hemat biaya untuk jaringan yang ditentukan perangkat lunak skala kecil," di *Proc. ICACT*, Pyeongchang, Korea Selatan, hlm. 758–761, 2014.

- [7] QH Nguyen, N. Ha Do dan H. Le, "Pengembangan switch berbasis SDN yang mampu menyediakan QoS dan hemat biaya untuk komunikasi IoT," di *Proc. ATC*, Kota Ho Chi Minh, hlm. 220–225, 2018.
- [8] V. Gupta, K. Kaur dan S. Kaur, "Mengembangkan switch jaringan yang ditentukan perangkat lunak berukuran kecil dan berbiaya rendah menggunakan raspberry pi," dalam *Next-Generation Networks*, Singapura: Springer, hlm. 147–152, 2018.
- [9] R. Bolla, C. Lombardo, R. Bruschi dan S. Mangialardi, "dropv2: Efisiensi energi melalui jaringan virtualisasi fungsi," *Jaringan IEEE*, vol. 28, tidak. 2, hal. 26–32, 2014.
- [10] BR Dawadi, DB Rawat, SR Joshi dan MM Keitsch, "Menuju efisiensi energi dan penerapan infrastruktur jaringan ramah lingkungan di Nepal menggunakan paradigma jaringan IPv6 yang ditentukan perangkat lunak," *The Electronic Journal of Information Systems in Developing Countries*, vol. 86, tidak. 1, hal. e12114, 2020.
- [11] MJ Piran, S. Verma, VG Menon dan DY Suh, "Model optimasi jangkauan transmisi hemat energi untuk internet of things berbasis wsn," *Computers, Materials & Continua*, vol. 67, tidak. 3, hal. 2989–3007, 2021.
- [12] S. Krishnamoorthy dan K. Narayanaswamy, "Alokasi dan penugasan pengontrol SDN berdasarkan algoritma salp gerombolan multikriteria yang kacau," *Intelligent Automation & Soft Computing*, vol. 27, tidak. 1, hlm.89–102, 2021.
- [13] I. Gravalos, P. Makris, K. Christodouloupolos dan EA Varvarigos, "Perencanaan jaringan yang efisien untuk internet of things dengan batasan QoS," *IEEE Internet of Things Journal*, vol. 5, tidak. 5, hal.3823–3836, 2018.
- [14] M. Beshley, N. Kryvinska, M. Seliuchenko, H. Beshley, EM Shakshuki *dkk.*, Algoritme manajemen "antrian cerdas" QoS ujung ke ujung dan mekanisme prioritas lalu lintas untuk layanan internet pita sempit di jaringan 4G/5G," *Sensor*, vol. 20, tidak. 8, hal.2324–2354, 2020.
- [15] S. Math, P. Tam dan S. Kim, "Pengaruh lalu lintas IoT real-time yang cerdas di jaringan edge 5G," *Komputer, Material & Continua*, vol. 67, tidak. 3, hal.3433–3450, 2021.
- [16] VS Naresh, SS Pericherla, P. Sita dan S. Reddi, "Internet dalam perawatan kesehatan: Arsitektur, aplikasi, tantangan, dan solusi," *Sains dan Teknik Sistem Komputer*, vol. 35, tidak. 6, hal.411–421, 2020.
- [17] H. Geng, J. Yao dan Y. Zhang, "Algoritme perlindungan perutean kegagalan tunggal di SDN hybrid jaringan," *Komputer, Material & Continua*, vol. 64, tidak. 1, hal.665–679, 2020.
- [18] O. Sadio, I. Ngom dan C. Lishou, "Desain dan pembuatan prototipe jaringan kendaraan yang ditentukan perangkat lunak," *IEEE Transactions on Vehicular Technology*, vol. 69, tidak. 1, hal.842–850, 2020.
- [19] DB Rawat dan SR Reddy, "Arsitektur jaringan yang ditentukan perangkat lunak, keamanan dan efisiensi energi: Sebuah survei," *IEEE Communications Surveys & Tutorials*, vol. 19, tidak. 1, hal.325–346, 2017.
- [20] BG Assefa dan Ö. Özkasap, "RESN: Metrik dan metode baru untuk perutean hemat energi dalam jaringan yang ditentukan perangkat lunak," *IEEE Transactions on Network and Service Management*, vol. 17, tidak. 2, hal.736–749, 2020.
- [21] A. Kannan, S. Vijayan, M. Narayanan dan M. Reddiar, "Mekanisme routing adaptif di SDN untuk membatasi kemacetan," *Desain Sistem Informasi dan Aplikasi Cerdas*, vol. 862, hlm.245–253, 2019. <https://doi.org/10.1007/978-981-13-3329-3>.
- [22] J. Wang, O. Kochan, K. Przystupa dan J. Su, "Sistem pengukuran informasi untuk mempelajari termokopel dengan medan suhu terkontrol," *Measurement Science Review*, vol. 19, tidak. 4, hlm.161–169, 2019.
- [23] H. Beshley, M. Kyryk, M. Beshley dan O. Panchenko, "Metode rekayasa arus informasi dan distribusi sumber daya dalam jaringan heterogen 4G/5G untuk penyediaan layanan M2M," di *Proc. IDAACS-SWS*, Lviv, Ukraina, hlm.229–233, 2018.
- [24] A. Pryslupskyi, O. Panchenko, M. Beshley dan M. Seliuchenko, "Peningkatan kinerja jaringan peralihan label multiprotokol menggunakan pengontrol yang ditentukan perangkat lunak," di *Proc. CADSM*, Polyana, Ukraina, hlm.106–109, 2019.
- [25] M. Beshley, M. Seliuchenko, O. Panchenko, O. Zyuzko dan I. Kahalo, "Analisis kinerja eksperimental dari saklar dan pengontrol jaringan yang ditentukan perangkat lunak," di *Proc. TCSET*, Lviv-Slavske, Ukraina, hlm. 282–286, 2018.

- [26] M. Beshley, V. Romanchuk, M. Seliuchenko dan A. Masiuk, "Investigasi metode antrian prioritas yang dimodifikasi berdasarkan pada test bed jaringan virtual," di *Proc.CADSM*, Lviv, Ukraina, hal. 1–4, 2015 .
- [27] S. Jun, K. Przystupa, M. Beshley, O. Kochan, H. Beshley *dkk.*, "Router dan generator lalu lintas berbasis perangkat lunak yang hemat biaya untuk simulasi dan pengujian jaringan IP," *Electronics*, vol. 9, tidak. 1, hal. 40–64, 2020.
- [28] B. Li dan S. Yu, "Penambangan kata kunci untuk protokol pribadi yang disalurkan melalui websocket," di *Komunitas IEEE Surat kation*, vol. 20, tidak. 7, hlm.1337–1340, 2016.
- [29] E. Mallada, X. Meng, M. Hack, L. Zhang dan A. Tang, "Sinkronisasi jam jaringan tanpa kemiringan tanpa diskontinuitas: Konvergensi dan kinerja," *Transaksi IEEE/ACM pada Jaringan*, vol. 23, tidak. 5, hlm.1619–1633, 2015.
- [30] J. Michałowska, A. Tofil, J. Jóźwik, J. Pytka, S. Legutko *dkk.*, "Memantau risiko komponen listrik yang dikenakan pada pilot selama operasi pesawat ringan di medan elektromagnetik frekuensi tinggi," *Sensor*, jilid. 19, tidak. 24, hal.5537, 2019.
- [31] R. Kochan, O. Kochan, M. Chyrka, S. Jun dan P. Bykovyy, "Pendekatan pengembangan pembagi tegangan untuk verifikasi metrologi ADC," di *Proc. IDAACS-2013*, Berlin, Jerman, vol. 1, hal.70–75, 2013.
- [32] DL Presti, C. Massaroni dan E. Schena, "Kisi-kisi serat optik untuk pengukuran kelembaban: A ulasan," *IEEE Sens. J.*, vol. 18, tidak. 22, hlm.9065–9074, 2018.
- [33] A. Glowacz, "Diagnosis kesalahan bor tumbukan listrik menggunakan pencitraan termal," *Pengukuran*, vol. 171, hal.108815, 2021.
- [34] S. Jun, O. Kochan, W. Chunzhi dan R. Kochan, "Penelitian teoritis dan eksperimental kesalahan metode termokopel dengan profil medan suhu terkontrol," *Meas. Sains. Pdt.*, jilid. 15, tidak. 6, hal.304–312, 2015.
- [35] AK Mallik, D. Liu, V. Kavungal, Q. Wu, G. Farrell *dkk.*, "Resonator mikro bola berlapis Agarose untuk pengukuran kelembapan," *Opt. Ekspres*, jilid. 24, tidak. 19, hal.21216–21227, 2016.
- [36] I. Kahalo, H. Beshley, M. Beshley dan O. Panchenko, "Meningkatkan qos dan efisiensi energi jaringan terintegrasi lte/lte-u/wi-fi berdasarkan teknik adaptif untuk pembentukan struktur radio," di *Proc. UKRCON*, Lviv, Ukraina, hlm. 1167–1170, 2019.
- [37] A. Montazerolghaem, MH Yaghmaee dan A. Leon-Garcia, "Jaringan multimedia cloud hijau: alokasi sumber daya hemat energi berbasis NFV/SDN," *IEEE Transactions on Green Communications and Networking*, vol. 4, tidak. 3, hal.873–889, 2020.
- [38] H. Zhu, X. Liao, C. de Laat dan P. Grosso, "Penjadwalan perutean aliran bersama untuk jaringan pusat data yang ditentukan perangkat lunak hemat energi: Sebuah prototipe platform manajemen jaringan yang sadar energi," *Jurnal Jaringan dan Komputer Aplikasi*, jilid. 63, hlm.110–124, 2016.
- [39] Y. Njah dan M. Cheriet, "Optimasi rute paralel dan jaminan layanan dalam perangkat lunak hemat energi-jaringan IoT industri yang ditentukan," *IEEE Access*, vol. 9, hal.24682–24696, 2021.